OCR Sudoku Solver

Rapport de première soutenance

Groupe 9 : DEGUEST Théodore, BONNEFONT Arthur, BONEFONT Thibaut, BOISARD Emile

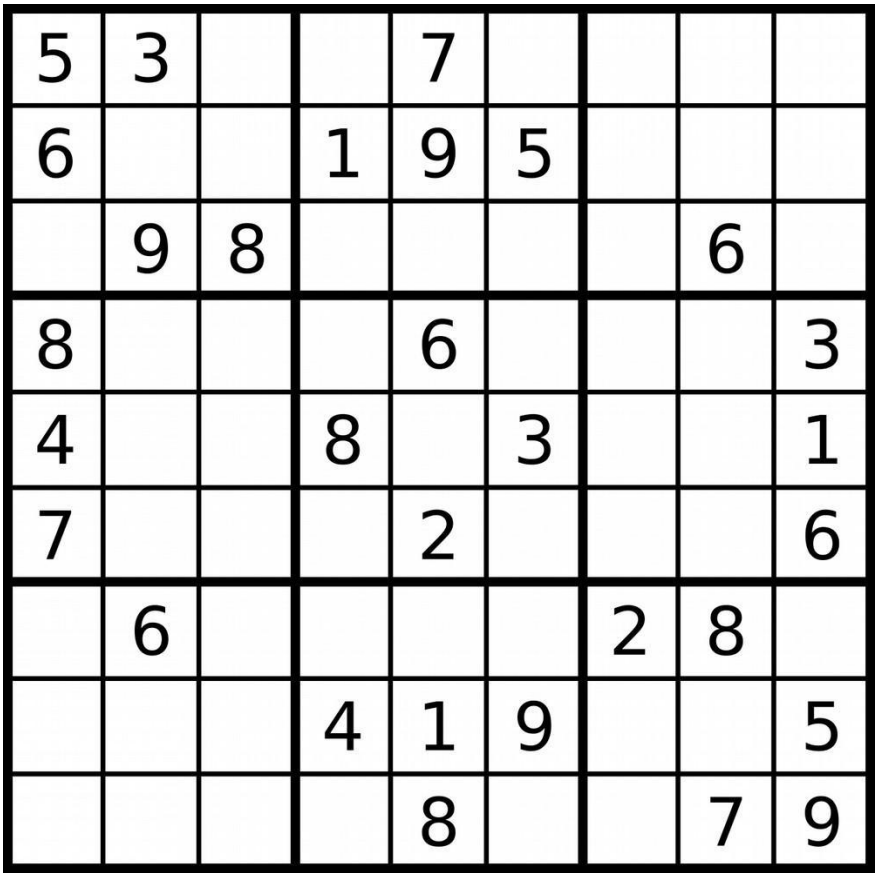


Table des matières

Répartition des tâches **3**

Avancement du projet **4**

Aspects technique **5**
- Chargement et suppression des couleurs d’une image 6
- Rotation manuelle d’une image 7
- Détection de la grille et de la position des cases 9
- Découpage d’une image 13
- Algorithme de résolution d’un sudoku 14
- Réseau de neurones 20

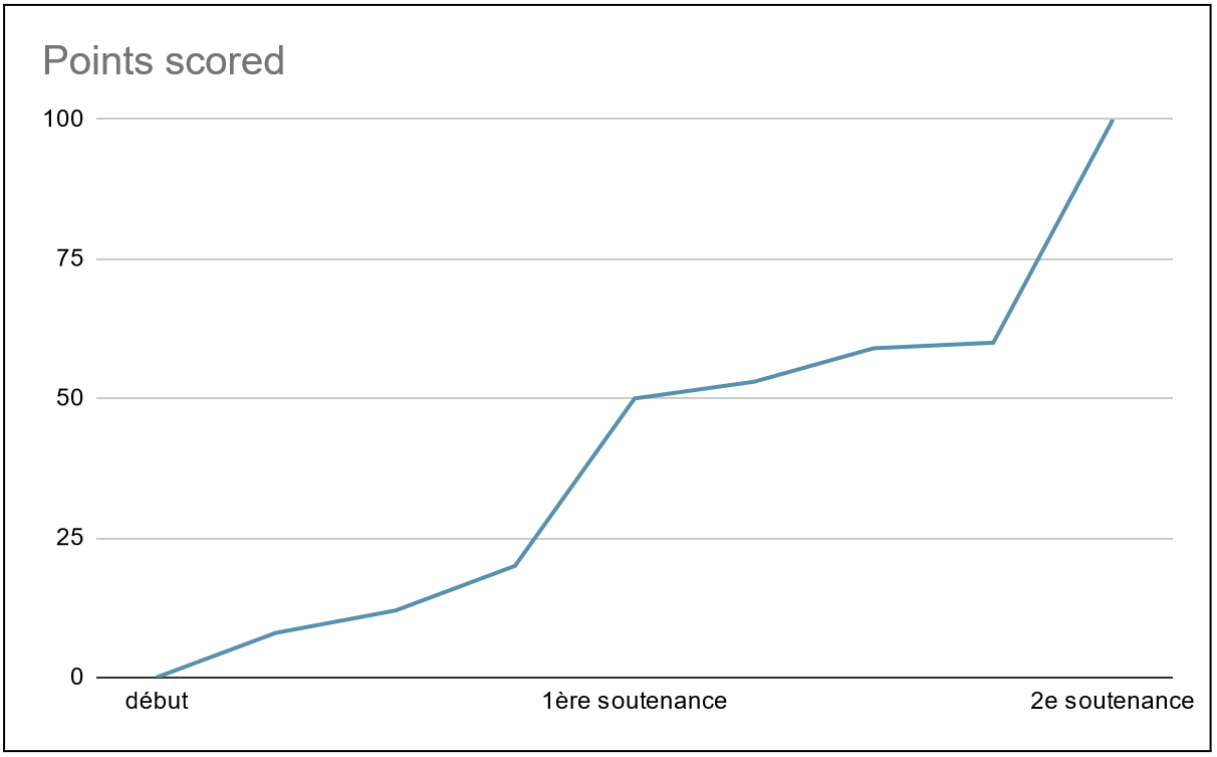
Conclusion **40**

Répartition des tâches

Cela a été difficile pour nous de mettre en place une répartition égale pour chaque élément du groupe que ce soit en termes de difficulté mais aussi de temps . En effet, nous avons eu du mal à évaluer le temps et la difficulté de chaque étape. Finalement, nous avons convenu de la répartition ci-dessous avec un fonctionnement de responsable et suppléant pour les tâches plus complexes.

Tâches	Théodore	Arthur	Thibault	Emile
Chargement d'une image et suppression de couleurs	Responsable		Suppléant	
Rotation manuelle de l'image				Responsable
Détection de la grille et de la position des cases	Responsable			Suppléant
Découpage de l'image		Suppléant		Responsable
algorithme de résolution d'un sudoku			Responsable	
Réseau de neurones		Responsable		
Interface graphique			Responsable	Suppléant

Avancement du projet



Chargement d’une image et suppression des couleurs :

- Le programme peut charger des images de divers types (jpg, png, bmp…)
- La suppression des couleurs est opérationnelle.

Rotation manuelle de l’image :

- La partie sur la rotation manuelle de l’image est terminée entièrement.

Détection de la grille et de la position des cases :

- Les grilles pré-orientées (sans inclinaison) et sans trop de motif autour sont détectées. Chaque case est récupérée et sa position dans la grille est connue.
- Les grilles avec beaucoup de bruit peuvent parfois engendrer des erreurs d’interprétation du programme.

Découpage de l’image :

- Le découpage de l’image est terminé. Nous avons réussi à découper les différentes cases sans avoir des bouts de lignes sur les différentes images.

Algorithme de résolution d’un sudoku :

- L’algorithme de résolution de sudoku est entièrement terminé que ce soit pour le 9*9 ou le 16*16.

Réseau de neurones

- Le réseau de neurones est théoriquement fonctionnel.

Aspects techniques

I/ Chargement d’une image et suppression des couleurs (Théodore)

Pour débuter le prétraitement, nous appliquons un flou gaussien et une suppression des couleurs.

Après avoir fait tout cela, les résultats étaient bons mais toujours pas parfaits. C’est pourquoi nous avons décidé d’améliorer encore notre pré-traitement en ajoutant différentes fonctions, qui pour le moment ne donnent pas les résultats attendus, et qui donc ne sont pas implémentées dans le projet.

La première mais également la plus basique est une gestion du contraste de l’image. Cette fonction prend en paramètre une surface sdl ainsi que le facteur de changement de contraste qui est un nombre décimal compris entre 0 et 2. Un facteur de 1 ne modifie pas l’image, un facteur supérieur à 1 augmente le contraste et un facteur inférieur à 1 diminue le contraste.

```
SDL_Surface* AdjustContrast(SDL_Surface* surface, float contrastFactor) {
    for (int y = 0; y < surface->h; y++) {
        for (int x = 0; x < surface->w; x++) {
           Uint8* pixel = (Uint8*)surface->pixels + y * surface->pitch + x *
                  surface->format->BytesPerPixel;

            // Apply the modifications
            for (int c = 0; c < surface->format->BytesPerPixel; c++) {
                pixel[c] = (Uint8)(contrastFactor * (pixel[c] - 128) + 128);
            }
        }
    }
    return surface;
}
```

Le deuxième ajout est une égalisation d’histogramme, à laquelle nous avons ajouté la possibilité de modifier le nombre de nuances de gris afin de limiter les artéfacts que peut créer ce traitement. Malheureusement, cet algorithme à montré des résultats inégaux en fonction des images prises en paramètre, avec parfois une correction de tous les défauts, et parfois la création de nouveaux défauts. C’est pourquoi nous avons décidé de ne pas l’utiliser pour le moment.

Le dernier ajout que nous avons fait est la normalisation de la luminosité de l'image. Cette fonction est sûrement la plus compliquée des trois à implémenter malgré que son principe soit assez simple. En effet, c'est un algorithme qui prend en paramètre une surface sdl ainsi qu'une luminosité et un écart type à atteindre. Le concept est donc de réduire la luminosité des zones plus claires que la luminosité cible et augmenter celle des zones plus sombres, en conservant toutes les valeurs de luminosité dans la zones permise par l'écart type

```
SDL_Surface* NormalizeImage(SDL_Surface* image, double targetLuminance, double targetStdDev);
```

Ces trois fonctions sont pour le moment à l'état de prototype mais nous comptons les utiliser dans la version finale du projet pour encore améliorer nos résultats. Cette partie est restée inchangée depuis la dernière soutenanvc.

II/ Rotation manuelle de l'image (Emile)

La rotation de l'image est une étape importante du projet. En effet, comme le prouve l'image 5 de l'annexe du cahier des charges, les grilles de sudoku ne sont pas forcément droites. Par conséquent, il est primordial de les recadrer pour simplifier les étapes suivantes, notamment la reconnaissance des chiffres par le réseau de neurones. Lors de la rotation d'une image il est important de garder toutes les données de l'image principale, il ne faut pas perdre des pixels, que les pixels changent de couleur ou bien même que l'image finale soit rognée.

Pour cela, nous avons créé une fonction "RotateSurface" qui prend en paramètres une surface SDL et un double, qui correspondent respectivement à l'image et à l'angle.

Tout d'abord, la fonction calcule les dimensions de l'image, ce qui va nous éviter d'avoir une image rognée après rotation grâce à ces lignes:

```
    double angle_radians = angle * M_PI / 180.0;
    int dest_width = (int)(original_width * fabs(cos(angle_radians)) + original_height *
fabs(sin(angle_radians)));
    int dest_height = (int)(original_height * fabs(cos(angle_radians)) + original_width *
fabs(sin(angle_radians)));
```

Ensuite elle crée une nouvelle surface(“rotated_surface”) à partir de la fonction “SDL_CreateRGBSurfaceWithFormat. Cette surface a les dimensions calculées précédemment et utilise le même format de pixels que la surface originale.

L’étape suivante est celle de l’obtention des couleurs pour cela on utilise cette fois-ci GetPixelRGBA. Grâce à ça, nous pouvons remplir la nouvelle surface avec ces couleurs. Puis, la fonction calcule les coordonnées de destination afin d’avoir une image centrée dans la nouvelle surface. Des rectangles source (src_rec) et destination (dst_rec) sont créés pour définir la région à copier depuis la surface originale et la région où la coller dans la nouvelle surface.

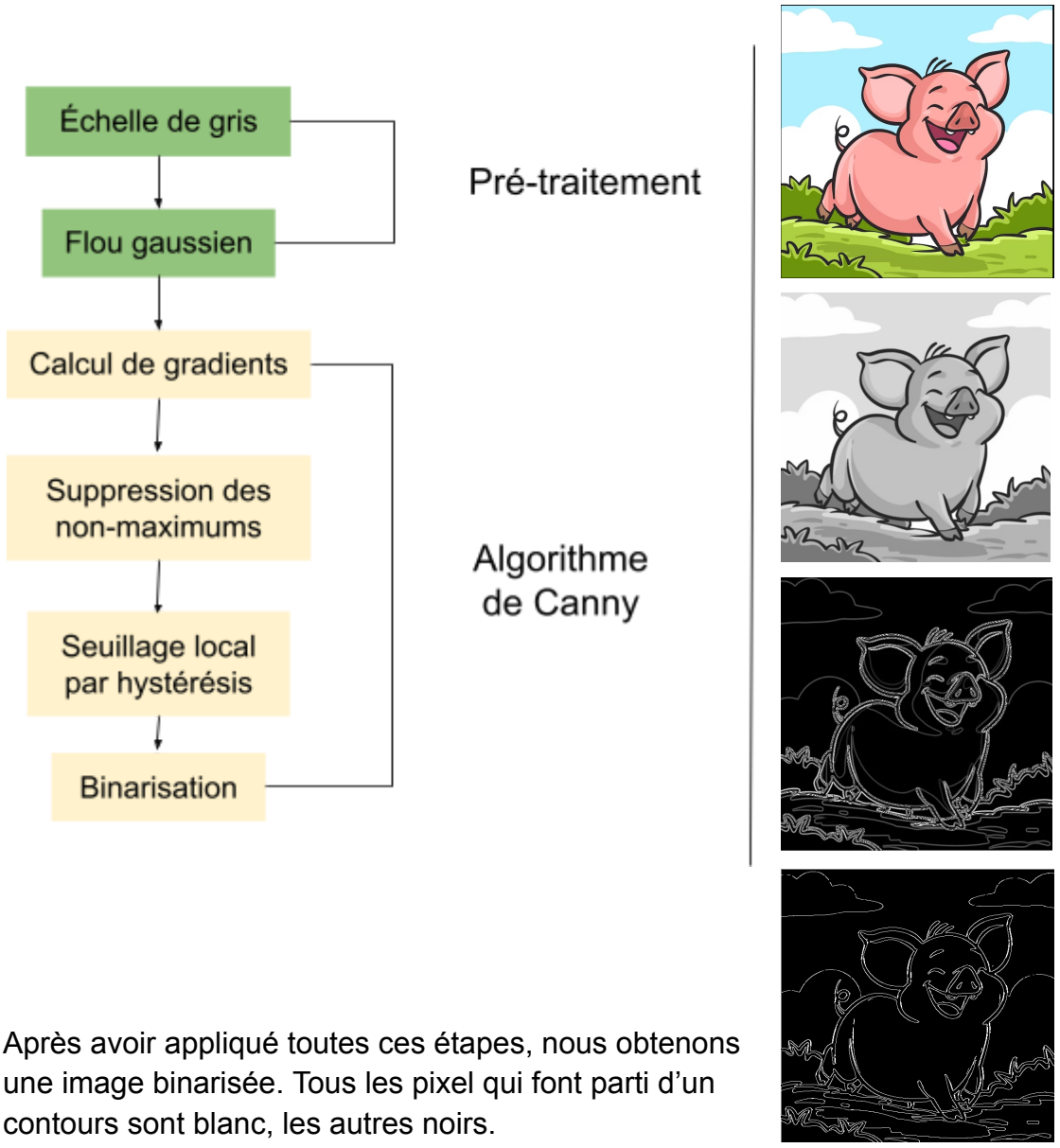
Après tout cela, nous pouvons enfin utiliser SDL_BlitSurfacepour effectuer la rotation. Enfin, La fonction renvoie la nouvelle surface rotative. Si la création de la nouvelle surface ou la rotation échoue, la fonction renvoie NULL.

III/ Détection de la grille et de la position des cases (Théodore)

Pour la première soutenance nous étions partis avec ce raisonnement :Afin de réaliser la détection de grille, notre algorithme est divisé en deux principales étapes. On utilise tout d’abord la méthode de Canny pour détecter les différents contours présents dans l’image, puis grâce à la transformée de Hough, on détecte

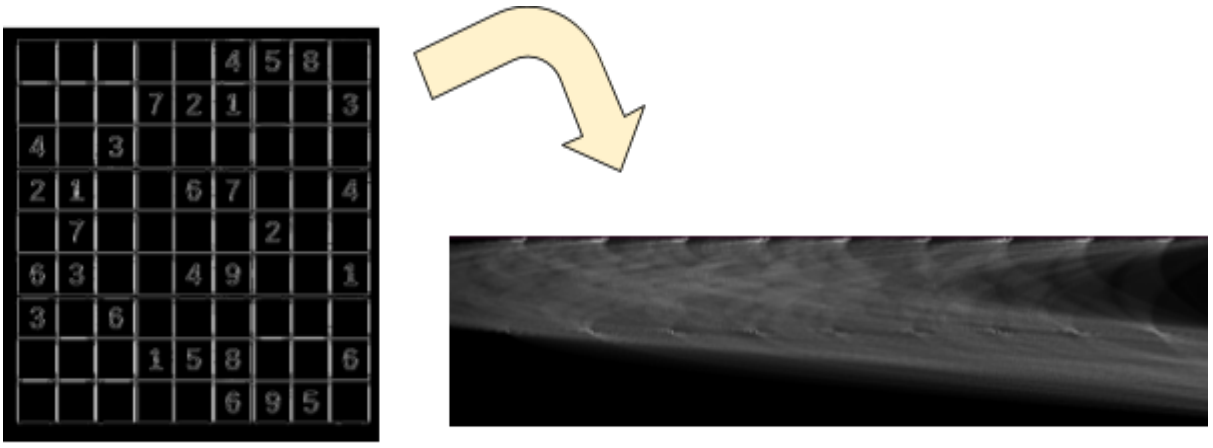
les différentes lignes présentes dans ces contours. Finalement nous exploitons ces données pour détecter la position des cases.

Pré-traitement : Afin d'optimiser les résultats obtenus par l'algorithme de canny, nous commençons par appliquer différents filtres.



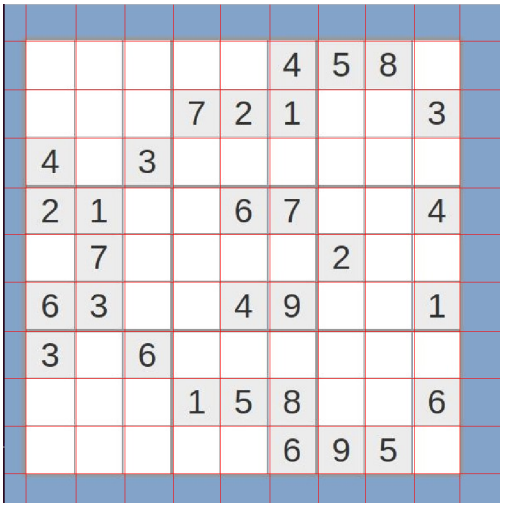
Après avoir appliqué toutes ces étapes, nous obtenons une image binarisée. Tous les pixel qui font parti d'un contours sont blanc, les autres noirs.

Détection des lignes: Une fois que nous avons trouvé les principaux contours de notre image, nous utilisons un accumulateur et la transformée de hough afin de regarder pour chaque combinaison paramètres (rho et theta) combien de points de contour sont sur la ligne résultante. Ainsi, nous obtenons une matrice qui représente les paramètres des droites dans le plan polaire.



```
for (int j = 0; j < n_y; j ++){
    for (int i = 0; i < n_x; i ++){
        // Check if pixel is white
        if (GetPixelLuminance(image_from_canny, i, j) > 0)
            for (int i_theta = 0; i_theta < n_theta; i_theta ++){
                double theta = i_theta * d_theta;
                double rho = i*cos(theta)+(n_y-j)*sin(theta);
                int i_rho = (int)(rho/d_rho);
                if (i_rho>0 && i_rho<n_rho)
                    accum[i_theta][i_rho] += 1;
            }
    }
}
```

(Avec ce code, nous passons d'un repère cartésien à une représentation polaire)



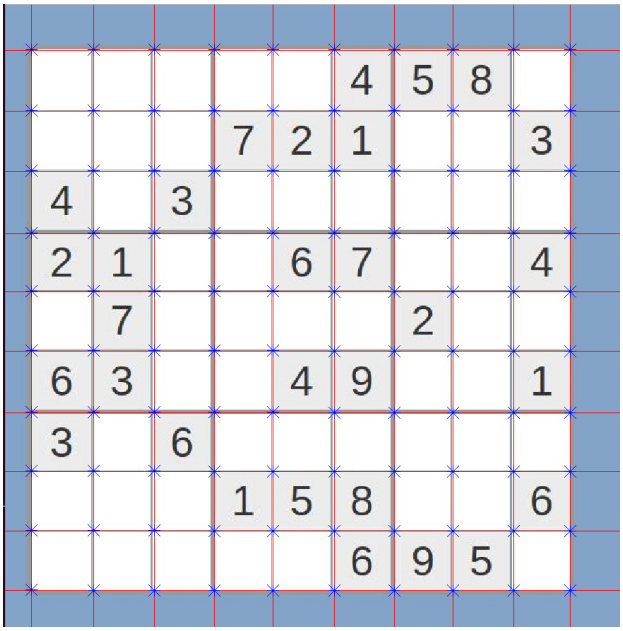
Une fois ceci fait, nous appliquons un seuillage afin de ne garder que les pixels les plus marqués de l'accumulateur, donc les lignes comportant le plus de points de contour.
Enfin nous parcourons de nouveau l'accumulateur et lorsque plusieurs pixel blanc se touchent, nous n'en gardons qu'un seul (on fait une moyenne des coordonées du groupe pour savoir lequel garder).
Ainsi, nous n'avons pas de lignes qui sont confondues.

(lignes détectées en rouge ->)

Détection des intersections: À partir des coordonnées polaires que nous avons obtenues, nous calculons pour chaque paire le point d'intersection.

```
        int x = (int)((polar_1.rho * sin(polar_2.theta) - polar_2.rho * sin(polar_1.theta)) /
sin(polar_2.theta - polar_1.theta));
        int y = n_y - (int)((polar_2.rho * cos(polar_1.theta) - polar_1.rho * cos(polar_2.theta)) /
sin(polar_2.theta - polar_1.theta));
```

Ainsi, nous testons pour chaque paire de ligne et nous obtenons tous les points d'intersections.



Pour en déduire les cases, nous partons du point le plus proche de l'origine puis nous regardons s'il y a des points à sa droite et en dessous de lui.
Dans ce cas, c'est le coins haut gauche d'une case. On récupère le point le plus proche à sa droite et en dessous de lui pour déduire les dimensions de la case, puis on passe au point suivant (à sa droite).
Sinon, on passe à la ligne suivante en regardant le point le plus proche de l'origine (donc le point le plus en haut à gauche des points restants).

Nous pouvons alors utiliser la fonction que nous avons créée pour récupérer une partie de l'image. Puisque notre algorithme itère de haut en bas et de gauche à droite, nous pouvons facilement les numéroter.


case_38.png


case_41.png


case_50.png


case_51.png

case_40.png

case_52.png

Enfin, pour améliorer l'interface, nous avons centralisé la gestion du solver, de la détection de grille et du xor dans le fichier main.

```
theodore@theodore-ubuntu:~/SudokuSolver$ ./main help
Usage: ./main <command> [<options>..]

Commands :
    canny: run canny filter
    solver: run sudoku solver
    xor: run xor neural network
    hough: run hough line detection
    grid: run grid detection detection (canny then hough)
    rotate: rotate an image
Options:
    -d: display image information
    -z: resize image (keep proportions), to fit in the screen (CANNY, HOUGH, GRID)
    -t: display processing time
    -o <filename>: store output in a file
    -i <filename>: choose input from a file
    -s: run step by step (CANNY, HOUGH, GRID)
    -h <value>: set high threshold to value (float) (CANNY, HOUGH, GRID)
    -l <value>: set low threshold to value (float) (CANNY, HOUGH, GRID)
    -b <value>: set blur straight to value (int) (CANNY, HOUGH, GRID)
    -x <value>: set nb iteration of xor to value * 10000 (int) (XOR)
    -p <value>: set displaying step of xor to value * 1000 (int) (XOR)
    -r <value>: set hough threshold to value (int) (HOUGH, GRID)
    -a <value>: set rotation angle to value (float) (ROTATE)
```

Ceci nous permet d'avoir la main sur de nombreux paramètres de nos différents sous programmes.

```
theodore@theodore-ubuntu:~/SudokuSolver$ ./main grid
0 : image_07.jpeg
1 : image_01.jpeg
2 : image_05.jpeg
3 : images_08.png
4 : pig.jpg
5 : image_09.jpg
6 : forme.png
Choose a file with index : 1
```

Cependant, pour cette deuxième et dernière soutenance, nous avons fait le choix de reprendre cette partie, tout simplement pour qu'elle soit plus efficace. Maintenant la détection de la grille repose sur le principe de la détection de masse.

IV/ Découpage de l'image (Emile)

Le découpage de l'image permet de simplifier le travail de reconnaissance des chiffres dans les différentes cases de la grille de sudoku. En effet, ce découpage permettra au réseau de neurones de se concentrer sur une case à la fois et donc un chiffre.

Pour cela, nous avons conçu la fonction `SaveSurfaceSample`,qui prend en paramètres une surface SDL (`source_surface`), les coordonnées `(x, y)` d'un échantillon à extraire de cette surface, ainsi que sa largeur (`width`) et sa hauteur (`height`). De plus, elle prend en entrée le nom du fichier dans lequel l'échantillon extrait sera enregistré au format PNG (file_name).

La fonction commence par vérifier si la surface source (`source_surface`) est valide. Si elle est nulle, la fonction renvoie `EXIT_FAILURE`, indiquant une erreur.

Ensuite, elle crée une nouvelle surface SDL (`sample_surface`) en utilisant les dimensions spécifiées (`width` et `height`). Cette surface sera utilisée pour stocker l'échantillon extrait de la surface source.

Après avoir créé la surface de destination, la fonction copie la zone spécifiée par les coordonnées `(x, y)` avec la largeur et la hauteur données depuis la surface source vers la surface de destination en utilisant la fonction `SDL_BlitSurface`. Comme le montre le code ci-dessous :

```
    SDL_Rect sample_rect = {x, y, width, height};
    if (SDL_BlitSurface(source_surface, &sample_rect, sample_surface, NULL) != 0)
        return EXIT_FAILURE;
```

Enfin, la fonction tente de sauvegarder la surface de destination au format PNG en utilisant la fonction `IMG_SavePNG`. Si cette opération échoue, la fonction libère la mémoire allouée pour la surface de destination et renvoie `EXIT_FAILURE`. Si la sauvegarde est réussie, la mémoire est également libérée, et la fonction renvoie `EXIT_SUCCESS`, indiquant que l'opération s'est déroulée avec succès.

V/ Algorithme de résolution d'un sudoku (Thibaut)

Pour l’algorithme de résolution du sudoku, nous avons décidé d’utiliser le backtracking. En effet, il nous a paru être la meilleure solution, car il est à la fois suffisamment optimisé pour résoudre n'importe quelle grille quasi instantanément mais également parce qu’il est relativement simple à comprendre et à implémenter. Le principe de base de cet algorithme est le suivant :

- déterminer la case à remplir, c’est-à-dire la première case vide sur notre chemin
- remplir cette case avec une valeur comprise entre 1 et 9 telle que cette valeur respecte les règles du sudoku
    - si une valeur qui respecte les règles du sudoku existe, on passe à la case suivante
    - sinon, on revient en arrière et on essaye d’autres valeurs jusqu’à en trouver une valide
- on répète récursivement ces deux étapes précédentes jusqu’à trouver un résultat valide si il en existe un

L’implémentation s’est donc faite de manière assez simple : nous avons la fonction récursive solve_sudoku ainsi que la fonction isValid que l’on appelle à chaque récursion afin de tester la valeur que nous essayons d’ajouter à la grille.

En ce qui concerne l’utilisation de ce programme, il vous suffit simplement d’appeler le fichier compilé en passant une grille de sudoku au format texte en paramètre. Pour ce faire, nous avons dû créer une fonction qui traduit ce fichier texte en une matrice d’int afin de pouvoir l’utiliser dans notre programme. Le résultat quand à lui peut être affiché dans la console comme demandé dans le cahier des charges et donc prendre cette forme ci:

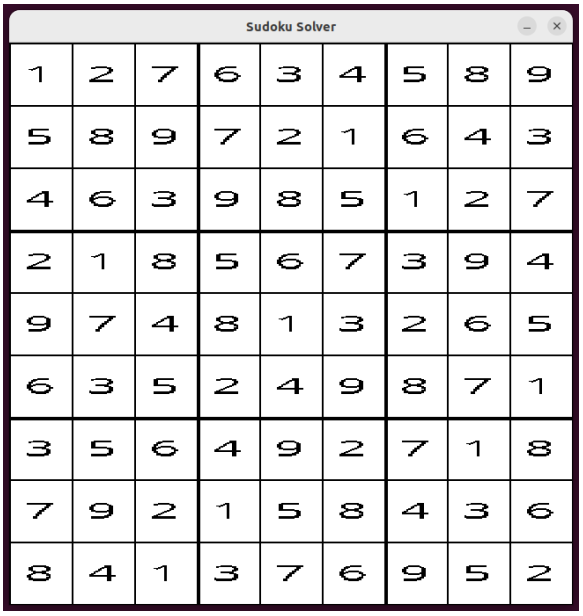
```
thigab@thigab-Lenovo-IdeaPad-S340-14IWL:~/epita/projetS3/SudokuSolver/editImageProcessing$ ./main solver
0 : grid_00
1 : grid_02
2 : grid_empty
3 : grid_01
Choose a file with index : 0
Chosen file is : grids/grid_00

 127 634 589
 589 721 643
 463 985 127

 218 567 394
 974 813 265
 635 249 871

 356 492 718
 792 158 436
 841 376 952
```

Mais nous avons choisi de rajouter également la possibilité d'afficher le résultat directement dans une image:



Pour cette deuxième soutenance, nous avons également réalisé un solveur de sudoku hexadécimal, aussi appelé Hexadoku, dont voici l'implémentation algorithmique :

-Le solveur utilise une approche de récursivité combinée avec des techniques de backtracking pour explorer les possibilités de remplissage de chaque cellule. Les fonctions allowedCol, allowedRow, et allowedBox sont cruciales pour vérifier la validité d'une valeur dans une colonne, une ligne, ou un bloc 4x4 respectivement.

```c
int allowedCol(char** grid, int col, char num);
int allowedRow(char** grid, int row, char num);
int allowedBox(char** grid, int boxRow, int boxCol, char num);
```

La construction des notes, réalisée par la fonction buildNotes, permet de stocker les possibilités pour chaque cellule vide, facilitant ainsi la recherche de solutions.

```c
void buildNotes(char** grid);
```

Les fonctions unique et singles appliquent des techniques de résolution avancées pour éliminer les possibilités et remplir les cellules avec une seule option valide, réduisant ainsi l'espace de recherche.

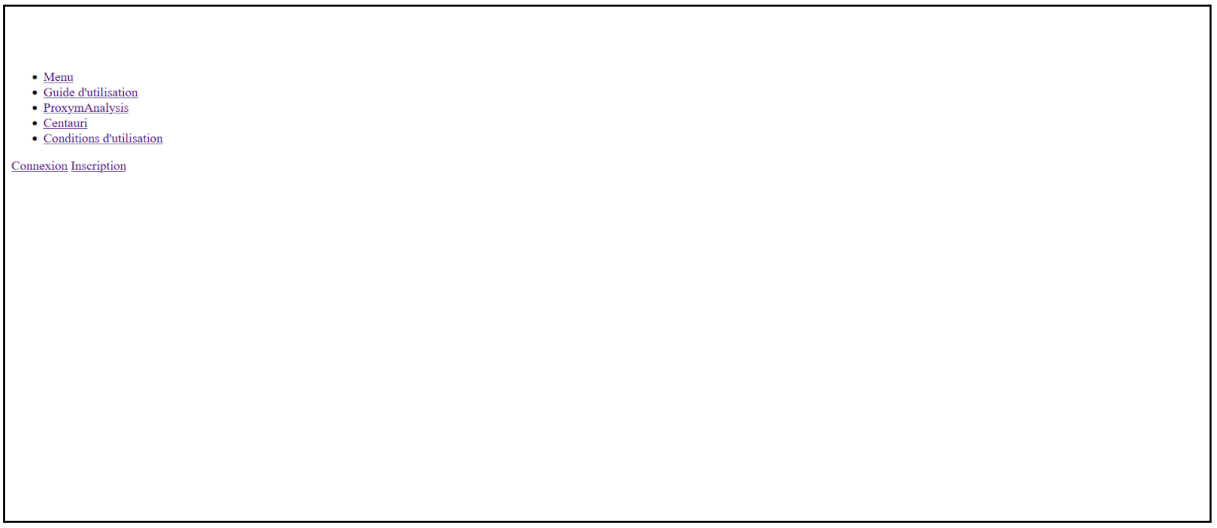
```c
int unique(char** grid);
int singles(char** grid);
```

Toutes ces fonctions nous ont permis d'améliorer drastiquement les performances de notre algorithme. En effet, en utilisant un backtracking simple comme on le fait pour les sudoku classiques, notre programme était parfaitement fonctionnel, mais il fallait au minimum 20 minutes à un ordinateur pour réussir à le résoudre, ce qui n'est pas un résultat satisfaisant selon nous.

VI/ Site Internet (Emile)

Nous avons fait le choix de faire un site internet pour ce projet, en effet, en réfléchissant aux différents bonus qui nous étaient proposés, le site internet était l'un des plus faciles et rapides à réaliser. Pour ce faire, nous avons repris un ancien site internet que nous avions conçu l'an dernier pour le projet du S2. Voici un résumé de la démarche que nous avions suivi :

“Afin que les utilisateurs aient accès au programme, nous avons fait le choix de faire un site internet. Ce dernier avait pour objectif de faciliter la manipulation du programme auprès des utilisateurs. En ce qui concerne l’architecture du site, nous avons fait le choix d’utiliser une barre de navigation pour que les utilisateurs puissent se déplacer aisément dans le site web. Elle est composée d’un menu pour retourner à la page principale du site, d’un guide d’utilisation pour aider les utilisateurs, de l’onglet « ProxymAnalysis » pour présenter le projet, de l’onglet « Centauri » pour présenter le groupe et des conditions d’utilisation. La page principale permettait d’être redirigée vers une page connexion ou une page inscription. La structure d’un site, codée en HTML, est importante puisqu’elle doit être au service du client afin qu’il puisse trouver les informations qu’il recherche rapidement.

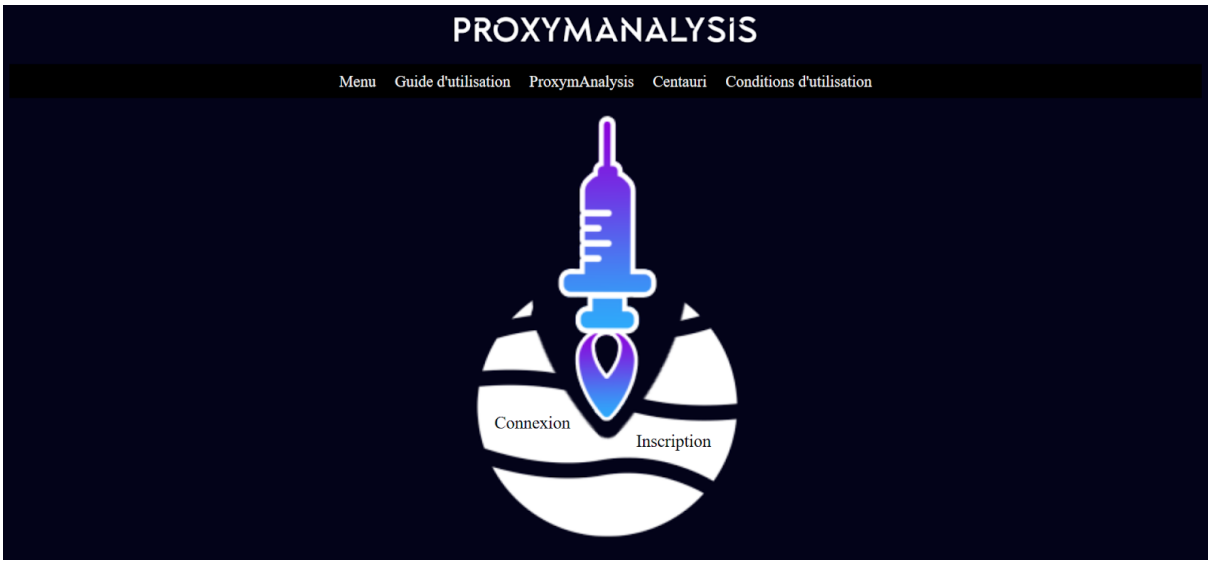


Capture d’écran de la page principale de site web sans css

1. Le style du site internet :

Dans un premier temps, en ce qui concerne le style, le code est principalement codé en CSS et est relié au HTML à partir de la balise <link> car nous avons fait le choix de mettre dans des pages différentes le CSS et le HTML pour des raisons de visibilité. Le nom du projet et le logo nous ont indiqués le thème à suivre pour le site web, ce thème n’est nul autre que l’espace. Pour cela, nous nous sommes mis d’accord sur le fait que le fond devait avoir une couleur sombre afin de respecter la thématique. De plus, le nom du projet et du groupe font référence à l’étoile Proxima du Centaure tout comme le logo. Cela fait naturellement penser à l’espace. L’objectif stylistique du site web était donc de respecter le plus possible l’idée de l’espace. Pour cela, nous avons commencé à chercher une couleur de fond rappelant l’espace. Après plusieurs essais, nous avons gardé une couleur bleu foncé proche du noir. Son code couleur en hexadécimal est #030319.

Un des objectifs majeurs du site est que l'utilisateur doit immédiatement reconnaître le projet. Le site doit donc avoir sa propre personnalité qui doit le différencier des autres afin qu'il puisse être immédiatement reconnu. Nous avions donc fait le choix de mettre le logo sur la page principale pour faciliter son identification. Pour amplifier celle-ci, nous avions placé le logo au milieu de la page pour que ce soit la première chose que les utilisateurs verront en visitant notre site internet. En outre, la position des boutons "inscriptions" et "connexion" n'était pas un hasard, nous les avions mis sur l'emblème du projet pour orienter l'œil du client.



Capture d'écran du site internet avec CSS

Nous avons fait l'ajout d'étoiles afin de rendre plus vivant et de dynamiser le site. Ces étoiles apparaissent et disparaissent de manière aléatoire. Pour l'aléatoire, nous avons dû utiliser du JavaScript pour pouvoir générer des nombres aléatoires et donc de pouvoir varier les positions et les temps d'apparitions. Contrairement au CSS, nous avons fait le choix d'intégrer le code JavaScript entièrement dans la page HTML puisqu'il n'était pas très long.

```
<script type="text/javascript">
        const starContainer = document.getElementById("star-container");
        const stars = document.getElementsByClassName("star");

        for (let i = 0; i < stars.length; i++) {
            stars[i].style.top = `${Math.random() * 100}%`;
            stars[i].style.left = `${Math.random() * 100}%`;
            stars[i].style.animationDelay = `${Math.random() * 20}s`;
            stars[i].addEventListener("animationiteration", () => {
                stars[i].style.top = `${Math.random() * 100}%`;
                stars[i].style.left = `${Math.random() * 100}%`;
            });
        }
    </script>
```

Capture d'écran du script javascript

.

Nous avons également fait l'ajout d'étoiles filantes en fond pour mieux représenter l'espace.

```
div.titre ul li:nth-child(1){
  animation-delay: 2s;
}
div.titre ul li:nth-child(2){
  animation-delay: 4s;
}
div.titre ul li:nth-child(3){
  animation-delay: 6s;
}
div.titre ul li:nth-child(4){
  animation-delay: 8s;
}
div.titre ul li:nth-child(5){
  animation-delay: 1s;
}
div.titre ul li:nth-child(6){
  animation-delay: 1.2s;
}
div.titre ul li:nth-child(7){
  animation-delay: 1.4s;
}
```

Une partie du code permettant d'animer le titre"

Après un ajustement du site voici à quoi ressemble la page principale de notre site internet pour le projet S3 :

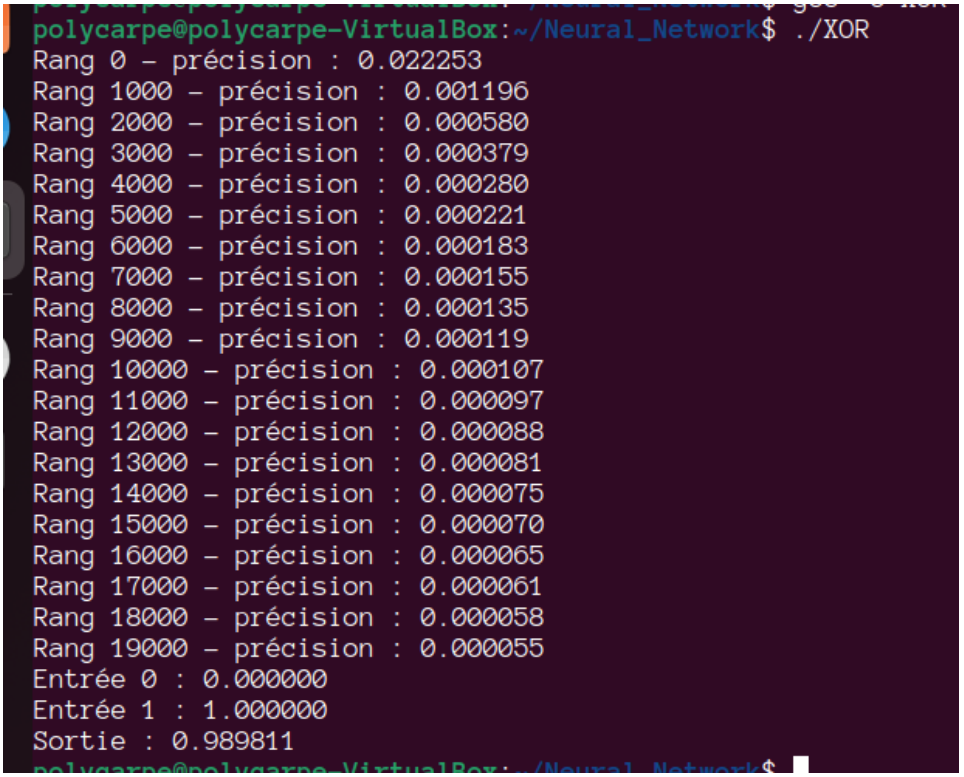


Page principale qui est la base du site

VII/ Réseau de neurones (Arthur)

En ce qui concerne le réseau de neurones, la première partie à été d'essayer de comprendre comment cela pouvait fonctionner, pour cela, nous avons commencé par lire le site qui nous avait été donné, cependant étant un peu indigeste à la première lecture nous nous sommes rabattus sur les millions de vidéos et de sites disponibles sur internet (celui d'IBM par exemple qui est plutôt bien expliqué). Une fois les bases mieux comprises, nous avons pu retourner au site donné et réaliser un premier test de réseau de neurones : **un réseau à deux neurones**.

Son but étant majoritairement de tester le principe de convergence des neurones, il était très simple et ne pouvait que répondre à l'input (0,1). Pour celui-ci, nous avons décidé d'utiliser la fonction sigmoïde (surtout par souci de simplicité) et parce qu'elle était connue pour marcher relativement bien. Après de nombreuses tentatives, de recherches pour faire compiler la librairie math.h", de segmentation fault et de programmes qui me renvoient des nombres aléatoires, nous y voilà enfin! Nous avions enfin des neurones qui marchaient, le programme renvoyait bien un nombre de plus en plus proche de 1 au fur et à mesure des essais successif.



(ce qui est appelé précision ici est simplement le taux d'erreur)

Le plus dur étant déjà passé, il ne restait plus qu'à adapter le code déjà existant pour qu'il ait 5 neurones et soit capable de résoudre un XOR. Pour cela, on a décidé de garder le sigmoïde et de faire une version qui transportait le résultat en nombre entier, cependant avec cette méthode, on ne pouvait pas observer la convergence, nous avons donc décidé de revenir avec la version en nombre flottant. Le code actuel renvoie donc un chiffre après la virgule.

Afin de simplifier le code, on a décidé de créer une structure « neurone » qui stocke les poids et le biais de chaque neurone. Pas forcément la plus utile pour un XOR avec 5 neurones mais qui deviendra indispensable pour un réseau avec plus de neurones. On a également décidé de laisser le choix à l'utilisateur de choisir le nombre d'essais, ainsi que l'écart d'affichage entre eux. Ce qui nous a posé le plus de problèmes ici à l'ajustement des poids des neurones, puisqu'on a dû l'adapter à la nouvelle structure de neurone.

Pour cette deuxième partie de projet, nous nous sommes donc inspirés de ce que nous avions fait auparavant pour concevoir le réseau de neurones traitant le xor. Le bagage intellectuel que nous avons pû acquérir lors de cette étape nous a grandement aidé pour réaliser le réseau de neurones final. Notre programme est découpé en sous partie comme ci-dessous :

- **Structure du réseau de neurones**

  La structure du réseau de neurones est définie dans la structure `NeuralNetwork`. Elle contient des informations sur le nombre de neurones en entrée, cachés et en sortie, ainsi que le taux d'apprentissage et les poids pour les couches cachées et de sortie.

```
const compose = (...fns) => res => fns.reduce((accum, next) => next(accum), res)

const unfold = (f, seed) => {
  const go = (f, seed, acc) => {
    const res = f(seed)
    return res ? go(f, res[1], acc.concat([res[0]])) : acc
  }
  return go(f, seed, [])
}
```

- **`input`**: Le nombre de neurones dans la couche d'entrée du réseau de neurones. Cela correspond généralement au nombre de fonctionnalités dans les données d'entrée.
- **`hidden`**: Le nombre de neurones dans la couche cachée du réseau de neurones. Cette couche est responsable de l'extraction de caractéristiques et de la représentation interne des données.
- **`output`**: Le nombre de neurones dans la couche de sortie du réseau de neurones. Il indique le nombre de classes ou de catégories que le réseau de neurones peut prédire.
- **`learning_rate`**: Le taux d'apprentissage utilisé lors de la mise à jour des poids du réseau de neurones pendant l'apprentissage. Il contrôle la taille des ajustements de poids à chaque itération d'apprentissage.

- **`hidden_weights`**: Une matrice représentant les poids entre la couche d'entrée et la couche cachée du réseau de neurones. Ces poids sont ajustés pendant l'apprentissage
- **`output_weights`**: Une matrice représentant les poids entre la couche cachée et la couche de sortie du réseau de neurones. Ces poids sont également ajustés pendant l'apprentissage.

En résumé, la structure du réseau de neurones stocke les paramètres essentiels du réseau, tels que le nombre de neurones dans chaque couche, le taux d'apprentissage, ainsi que les matrices de poids qui déterminent comment les informations sont propagées à travers le réseau lors de l'apprentissage et de la prédiction.

- **Création du réseau**

  La fonction `network_create` initialise un réseau de neurones avec des poids aléatoires. Elle alloue de la mémoire pour la structure du réseau, crée des matrices pour les couches cachées et de sortie, et les initialise avec des valeurs aléatoires.

```c
NeuralNetwork* network_create(int input, int hidden, int output, double lr) {
    NeuralNetwork* net = malloc(sizeof(NeuralNetwork)); // Allocation de mémoire pour la structure du réseau

    // Initialisation des paramètres du réseau
    net->input = input;
    net->hidden = hidden;
    net->output = output;
    net->learning_rate = lr;

    // Création de la matrice des poids pour la couche cachée
    Matrix* hidden_layer = matrix_create(hidden, input);

    // Création de la matrice des poids pour la couche de sortie
    Matrix* output_layer = matrix_create(output, hidden);

    // Initialisation des matrices de poids avec des valeurs aléatoires
    matrix_randomize(hidden_layer, hidden);
    matrix_randomize(output_layer, output);

    // Attribution des matrices de poids au réseau
    net->hidden_weights = hidden_layer;
    net->output_weights = output_layer;

    return net; // Retourne le réseau initialisé
}
```

En résumé, la fonction `network_create` effectue les étapes suivantes :

1. Alloue de la mémoire pour la structure `NeuralNetwork`.
2. Initialise les paramètres du réseau, tels que le nombre de neurones dans chaque couche et le taux d'apprentissage.
3. Crée les matrices de poids pour les couches cachées et de sortie en utilisant la fonction `matrix_create`.
4. Initialise les valeurs des matrices de poids avec des valeurs aléatoires en utilisant la fonction `matrix_randomize`.
5. Attribution des matrices de poids nouvellement créées à la structure du réseau.

Le résultat est un réseau de neurones initialisé avec des poids aléatoires, prêt à être utilisé pour l'apprentissage et la prédiction.

- **Propagation avant**

La propagation avant (forward propagation) est la phase d'évaluation d'un réseau de neurones où les données d'entrée sont introduites dans le réseau, et les calculs sont effectués couche par couche jusqu'à ce que la sortie finale soit obtenue. Dans le contexte de ce code, la propagation avant est réalisée par la fonction `net`

```c
void network_train(NeuralNetwork* net, Matrix* input, Matrix* output) {
    // Feed forward
    Matrix* hidden_inputs   = dot(net->hidden_weights, input);
    Matrix* hidden_outputs = apply(sigmoid, hidden_inputs);
    Matrix* final_inputs = dot(net->output_weights, hidden_outputs);
    Matrix* final_outputs = apply(sigmoid, final_inputs);

    // Find errors
    Matrix* output_errors = subtract(output, final_outputs);
    Matrix* transposed_mat = transpose(net->output_weights);
    Matrix* hidden_errors = dot(transposed_mat, output_errors);
    matrix_free(transposed_mat);

    Matrix* sigmoid_primed_mat = sigmoidPrime(final_outputs);
    Matrix* multiplied_mat = multiply(output_errors, sigmoid_primed_mat);
    transposed_mat = transpose(hidden_outputs);
    Matrix* dot_mat = dot(multiplied_mat, transposed_mat);
    Matrix* scaled_mat = scale(net->learning_rate, dot_mat);
    Matrix* added_mat = add(net->output_weights, scaled_mat);

    matrix_free(net->output_weights); // Free the old weights before replacing
    net->output_weights = added_mat;

    matrix_free(sigmoid_primed_mat);
    matrix_free(multiplied_mat);
    matrix_free(transposed_mat);
    matrix_free(dot_mat);
    matrix_free(scaled_mat);

    sigmoid_primed_mat = sigmoidPrime(hidden_outputs);
    multiplied_mat = multiply(hidden_errors, sigmoid_primed_mat);
    transposed_mat = transpose(input);
    dot_mat = dot(multiplied_mat, transposed_mat);
    scaled_mat = scale(net->learning_rate, dot_mat);
    added_mat = add(net->hidden_weights, scaled_mat);
    matrix_free(net->hidden_weights); // Free the old hidden_weights before replacement
    net->hidden_weights = added_mat;

    matrix_free(sigmoid_primed_mat);
    matrix_free(multiplied_mat);
    matrix_free(transposed_mat);
    matrix_free(dot_mat);
    matrix_free(scaled_mat);

    // Free matrices
    matrix_free(hidden_inputs);
    matrix_free(hidden_outputs);
    matrix_free(final_inputs);
    matrix_free(final_outputs);
    matrix_free(output_errors);
    matrix_free(hidden_errors);
}
```

`work_train` et `network_predict` qui effectuent les étapes suivantes :

1. **Calcul des sorties des couches cachées et de sortie :**
    - Calcule `hidden_inputs` en multipliant les poids de la couche cachée (`net->hidden_weights`) par les données d'entrée (`input`) à l'aide de la fonction `dot`.
    - Applique la fonction d'activation sigmoïde (`sigmoid`) à `hidden_inputs` pour obtenir `hidden_outputs`.
    - Calcule `final_inputs` en multipliant les poids de la couche de sortie (`net->output_weights`) par `hidden_outputs`.
    - Applique la fonction d'activation sigmoïde à `final_inputs` pour obtenir `final_outputs`.

2. **Calcul des erreurs :**
    - Calcule les erreurs de sortie en soustrayant la sortie prédite (`final_outputs`) de la sortie réelle (`output`).

3. **Rétropropagation :**
    - Calcule les erreurs de la couche cachée en utilisant la transposée des poids de la couche de sortie (`transposed_mat = transpose(net->output_weights)`), puis en effectuant un produit matriciel avec les erreurs de sortie (`hidden_errors = dot(transposed_mat, output_errors)`).

4. **Mise à jour des poids de la couche de sortie :**
    - Calcule la dérivée de la fonction d'activation (`sigmoidPrime`) pour la couche de sortie.
    - Multiplie cette dérivée par les

- **Rétropropagation**

La rétropropagation (backpropagation) est une phase essentielle de l'apprentissage des réseaux de neurones, où les erreurs calculées lors de la propagation avant sont utilisées pour ajuster les poids du réseau afin de minimiser ces erreurs. Dans le contexte de ce code, la rétropropagation est implémentée dans la fonction `network_train`. Voici les principales étapes de la rétropropagation :

1. **Calcul des erreurs :**

- Les erreurs de sortie sont calculées en soustrayant la sortie prédite du réseau (`final_outputs`) de la sortie réelle (`output`). Cela donne une mesure de l'écart entre la prédiction du réseau et la vérité terrain.

2. **Calcul des erreurs de la couche cachée :**

- Les erreurs de la couche cachée sont calculées en multipliant la transposée des poids de la couche de sortie (`transposed_mat = transpose(net->output_weights)`) par les erreurs de sortie. Cela permet de propager les erreurs de sortie vers la couche cachée.

3. **Calcul des gradients locaux :**

- Les gradients locaux sont calculés en multipliant les erreurs de chaque couche par la dérivée de la fonction d'activation de cette couche. Dans ce code, la fonction d'activation utilisée est la fonction sigmoïde (`sigmoid`). Cela produit le gradient local qui indique comment les poids de chaque neurone doivent être ajustés pour minimiser l'erreur.

4. **Mise à jour des poids de la couche de sortie :**

- Les poids de la couche de sortie sont mis à jour en ajoutant à ces poids le produit de trois matrices : le taux d'apprentissage (`net->learning_rate`), le produit terme à terme de l'erreur de sortie et de la dérivée de la fonction sigmoïde appliquée à la sortie de la couche de sortie, et la transposée des sorties de la couche cachée.

5. **Mise à jour des poids de la couche cachée :**

- Les poids de la couche cachée sont mis à jour de manière similaire en utilisant les erreurs de la couche cachée, la dérivée de la fonction sigmoïde appliquée à la sortie de la couche cachée, et les données d'entrée.

6. **Libération de la mémoire :**

- La mémoire des matrices temporaires est libérée pour éviter les fuites de mémoire.

La rétropropagation permet au réseau de neurones d'ajuster ses poids de manière itérative, en utilisant les erreurs calculées pour améliorer progressivement sa capacité à produire des prédictions précises. C'est un processus clé dans l'apprentissage supervisé des réseaux de neurones..

- **Entraînement par lots**

L'entraînement par lots (batch training) est une technique couramment utilisée dans l'apprentissage des réseaux de neurones. Plutôt que de mettre à jour les poids du réseau après chaque exemple d'entraînement, on divise l'ensemble de données d'entraînement en lots (ou mini-lots) et on met à jour les poids après chaque lot. Dans le contexte de ce code, l'entraînement par lots est réalisé par la fonction `network_train_batch_imgs`.

Voici comment cela fonctionne :

```
void network_train_batch_imgs(NeuralNetwork* net, Img** imgs, int batch_size) {
    for (int i = 0; i < batch_size; i++) {
        if (i % 100 == 0) printf("Img No. %d\n", i);
        Img* cur_img = imgs[i];
        Matrix* img_data = matrix_flatten(cur_img->img_data, 0); // 0 = flatten to column vector
        Matrix* output = matrix_create(net->output, 1);
        output->entries[cur_img->label][0] = 1; // Setting the result
        network_train(net, img_data, output);
        matrix_free(output);
        matrix_free(img_data);
    }
}
```

1. **Boucle sur chaque lot :**
    - La fonction `network_train_batch_imgs` prend en paramètre un tableau d'images (`imgs`) et la taille du lot (`batch_size`).
    - Elle utilise une boucle pour itérer sur chaque lot d'images dans le tableau.
2. **Prétraitement des images :**
    - Pour chaque image dans le lot, la fonction :
        - Aplatit les données de l'image en une matrice colonne en utilisant `matrix_flatten`.
        - Crée une matrice colonne `output` représentant la sortie désirée pour l'image. Par exemple, pour une tâche de classification, cela peut être un vecteur où la classe correspondante à l'image est marquée comme 1 et les autres comme 0.
3. **Appel à `network_train` :**
    - La fonction `network_train` est appelée avec la matrice d'entrée et la matrice de sortie préparées pour chaque image du lot.
    - `network_train` effectue la propagation avant, la rétropropagation et ajuste les poids du réseau en fonction des erreurs calculées.
4. **Libération de la mémoire :**
    - La mémoire des matrices temporaires (notamment les matrices d'entrée et de sortie) est libérée après chaque lot pour éviter les fuites de mémoire.

L'entraînement par lots présente plusieurs avantages, notamment l'optimisation du calcul matriciel (efficacité de calcul sur les processeurs modernes) et la stabilisation de la convergence du réseau. Cela permet également de tirer parti de la parallélisation sur les GPU pour accélérer le processus d'entraînement.

En résumé, l'entraînement par lots est une technique d'optimisation qui permet de mettre à jour les poids du réseau après avoir traité plusieurs exemples d'entraînement en même temps, améliorant ainsi l'efficacité et la stabilité de l'apprentissage.

- **Prédiction**

La prédiction dans le contexte des réseaux de neurones consiste à utiliser un modèle de réseau de neurones entraîné pour estimer la sortie correspondante à de nouvelles données d'entrée, sans ajuster les poids du réseau pendant ce processus. Dans le code que vous avez fourni, la fonction `network_predict` effectue cette opération.

```
Matrix* network_predict_img(NeuralNetwork* net, Img* img) {
    Matrix* img_data = matrix_flatten(img->img_data, 0);
    Matrix* res = network_predict(net, img_data);
    matrix_free(img_data);
    return res;
}
```

1. **Propagation avant pour la prédiction :**

- La fonction `network_predict` effectue la propagation avant du réseau, ce qui signifie qu'elle utilise les poids du réseau pour calculer la sortie du réseau en fonction des nouvelles données d'entrée fournies.

- Elle utilise les poids de la couche cachée (`net->hidden_weights`) et de la couche de sortie (`net->output_weights`) pour calculer les sorties des couches cachées et de sortie respectivement.

2. **Calcul de la probabilité de classe :**

- Une fois que la propagation avant est effectuée, la fonction applique la fonction softmax à la sortie de la couche de sortie. Cela produit une distribution de probabilités sur les différentes classes, indiquant la probabilité estimée de chaque classe.

3. **Libération de la mémoire :**

- La mémoire des matrices temporaires utilisées pour les calculs est libérée pour éviter les fuites de mémoire.

4. **Retour du résultat :**

- La fonction renvoie la matrice résultante, qui représente la probabilité estimée de chaque classe. Cette matrice peut ensuite être interprétée pour déterminer la classe prédite.

En résumé, la prédiction consiste à utiliser les poids déjà entraînés d'un réseau de neurones pour calculer la sortie estimée en réponse à de nouvelles données d'entrée. Cette sortie est souvent interprétée comme une distribution de probabilités sur les différentes classes, et la classe prédite est généralement celle avec la probabilité la plus élevée. La fonction `network_predict` encapsule ce processus dans le code que vous avez fourni.

- **Sauvegarde et chargement**

La sauvegarde et le chargement dans le contexte de ce code se réfèrent à la possibilité de sauvegarder les paramètres d'un réseau de neurones (poids des connexions, architecture, etc.) dans des fichiers, et de recharger ces paramètres à partir de ces fichiers pour réutiliser le modèle sauvegardé ultérieurement. Ces opérations sont généralement utiles pour conserver les résultats d'un entraînement prolongé ou pour partager des modèles avec d'autres utilisateurs

```
void network_save(NeuralNetwork* net, char* file_string) {
    mkdir(file_string, 0777);
    // Write the descriptor file
    chdir(file_string);
    FILE* descriptor = fopen("descriptor", "w");
    fprintf(descriptor, "%d\n", net->input);
    fprintf(descriptor, "%d\n", net->hidden);
    fprintf(descriptor, "%d\n", net->output);
    fclose(descriptor);
    matrix_save(net->hidden_weights, "hidden");
    matrix_save(net->output_weights, "output");
    printf("Successfully written to '%s'\n", file_string);
    chdir("-"); // Go back to the orignal directory
}
```

La fonction `network_save` sauvegarde les paramètres d'un réseau de neurones dans un répertoire spécifié. Voici comment cela fonctionne :

1. **Création d'un répertoire :**
   - La fonction crée un répertoire avec le nom spécifié (`file_string`) en utilisant `mkdir`. Ce répertoire sera utilisé pour stocker les fichiers de sauvegarde du réseau.
2. **Écriture d'un fichier descripteur :**
   - La fonction ouvre un fichier appelé "descriptor" dans le répertoire et écrit les informations importantes du réseau, telles que le nombre de neurones dans chaque couche (entrée, cachée, sortie). Ces informations seront utilisées lors du chargement du réseau.
3. **Sauvegarde des poids des couches :**
   - La fonction utilise les fonctions `matrix_save` pour sauvegarder les poids de la couche cachée (`hidden_weights`) et de la couche de sortie (`output_weights`) dans des fichiers distincts ("hidden" et "output" respectivement).

```c
NeuralNetwork* network_load(char* file_string, int hexa) {
    NeuralNetwork* net = malloc(sizeof(NeuralNetwork));
    char entry[MAXCHAR];
    chdir(file_string);

    FILE* descriptor;
    if (hexa == 0)
     descriptor = fopen("descriptor", "r");
    else
        descriptor = fopen("descriptor_hexa", "r");

    fgets(entry, MAXCHAR, descriptor);
    net->input = atoi(entry);
    fgets(entry, MAXCHAR, descriptor);
    net->hidden = atoi(entry);
    fgets(entry, MAXCHAR, descriptor);
    net->output = atoi(entry);
    fclose(descriptor);
    if (hexa == 0){
        net->hidden_weights = matrix_load("hidden");
        net->output_weights = matrix_load("output");
        }
    else{
        net->hidden_weights = matrix_load("hidden_hexa");
        net->output_weights = matrix_load("output_hexa");
    }
    printf("Successfully loaded network from '%s'\n", file_string);
    chdir("-"); // Go back to the original directory
    return net;
}
```

La fonction `network_load` charge un réseau de neurones à partir des fichiers sauvegardés. Voici comment cela fonctionne :

- **Changement de répertoire**
  - La fonction change le répertoire de travail vers le répertoire spécifié (`file_string`) où les fichiers de sauvegarde du réseau sont stockés.
- **Lecture d'un fichier descripteur :**
  - La fonction ouvre un fichier appelé "descriptor" et lit les informations du réseau, telles que le nombre de neurones dans chaque couche. Ces informations sont utilisées pour créer la structure du réseau.
- **Chargement des poids des couches :**
  - La fonction utilise les fonctions `matrix_load` pour charger les poids de la couche cachée (`hidden_weights`) et de la couche de sortie (`output_weights`) à partir des fichiers sauvegardés ("hidden" et "output" respectivement).
- **Retour du réseau chargé :**
  - La fonction retourne un pointeur vers la structure du réseau de neurones chargé.
- **Changement de répertoire (retour à l'original) :**
  - Après le chargement, la fonction change le répertoire de travail pour revenir à l'emplacement original.

- ## Prédiction sur un lot d'images

  La fonction `network_predict_imgs` dans le code que vous avez fourni effectue la prédiction sur un lot d'images. Voici comment cela fonctionne :

```c
double network_predict_imgs(NeuralNetwork* net, Img** imgs, int n) {
    int n_correct = 0;
    for (int i = 0; i < n; i++) {
        Matrix* prediction = network_predict_img(net, imgs[i]);
        if (matrix_argmax(prediction) == imgs[i]->label) {
            n_correct++;
        }
        int id = matrix_argmax(prediction);
        printf("%d, %.4lf\n", id, matrix_score(prediction, id));
        matrix_free(prediction);
    }
    return 100.0 * n_correct / n;
}
```

1. **Boucle sur chaque image du lot :**

- La fonction `network_predict_imgs` prend en paramètre un tableau d'images (`imgs`) et le nombre d'images dans le lot (`n`).

- Elle utilise une boucle pour itérer sur chaque image dans le tableau.

2. **Prétraitement de l'image :**

- Pour chaque image dans le lot, la fonction crée une matrice d'entrée `img_data` en aplatissant les données de l'image à l'aide de `matrix_flatten`. Cela crée une représentation sous forme de vecteur de colonnes de l'image.

3. **Appel à `network_predict_img` :**

- La fonction `network_predict_img` est appelée avec la matrice d'entrée `img_data` pour obtenir la prédiction pour cette image spécifique.

4. **Comparaison avec l'étiquette réelle :**

- La fonction compare la classe prédite avec l'étiquette réelle de l'image. Si la classe prédite correspond à l'étiquette réelle, le nombre de prédictions correctes (`n_correct`) est incrémenté.

5. **Affichage des résultats (débogage) :**

- La fonction affiche l'identifiant de la classe prédite (`id`) et la probabilité associée, utilisant `matrix_score`.

6. **Libération de la mémoire :**

- La mémoire des matrices temporaires (notamment `img_data` et `prediction`) est libérée après chaque prédiction pour éviter les fuites de mémoire.

7. **Calcul du pourcentage de prédictions correctes :**

- La fonction retourne le pourcentage de prédictions correctes sur l'ensemble du lot.

En résumé, la prédiction sur un lot d'images consiste à utiliser un réseau de neurones déjà entraîné pour prédire la classe de chaque image dans le lot. Les résultats sont ensuite comparés avec les étiquettes réelles pour évaluer la précision du modèle sur ce lot particulier. La fonction `network_predict_imgs` encapsule ce processus dans le code que vous avez fourni.

- **Évaluation des performances du réseau de neurones**

L'évaluation des performances d'un réseau de neurones consiste à mesurer sa capacité à effectuer des prédictions précises sur un ensemble de données. Dans le contexte du code que vous avez fourni, l'évaluation des performances se fait principalement à travers la fonction `network_predict_imgs` et peut également impliquer l'utilisation d'autres fonctions telles que `matrix_argmax` et `matrix_score`. Voici comment cela fonctionne :

1. **Boucle sur chaque image du lot :**

- La fonction `network_predict_imgs` utilise une boucle pour itérer sur chaque image dans le tableau d'images (`imgs`).

2. **Appel à `network_predict_img` :**

- Pour chaque image, la fonction appelle `network_predict_img` pour obtenir la prédiction du réseau.

```c
void perform_nn(int hexa, int res[][16]) {

    srand(time(NULL));

    Img** imgs = NULL;
    NeuralNetwork* net = NULL;

    imgs = calloc(300, 8);
    DIR* dir = opendir("./output");

    if (dir == NULL) {
        perror("Error opening directory");
        exit(EXIT_FAILURE);
    }

    int count = 0;
    struct dirent* entry;
    //char* paths[260];
    FileImagePair* fileImagePairs = malloc(260 * sizeof(FileImagePair));
    while ((entry = readdir(dir)) != NULL) {
        // Ignore les entrées spéciales "." et ".."
        if (strcmp(entry->d_name, ".") != 0 && strcmp(entry->d_name, "..") != 0) {
            // Construit le chemin complet du fichier
            char filepath[265]; // Assurez-vous que cela convient à vos besoins
            snprintf(filepath, sizeof(filepath), "%s/%s", "output", entry->d_name);
            //printf("%s\n", filepath);
            fileImagePairs[count].filename = strdup(filepath);
            bmp_to_csv(filepath, "tmp.csv");
            Img** tmp = csv_to_imgs("tmp.csv", 1);
            fileImagePairs[count].img = tmp[0];
            //imgs[count] = tmp[0];
            //img_print(imgs[count]);
            //imgs_free(tmp, 1);
            count ++;
        }
    }

    // Trie les noms de fichiers
    qsort(fileImagePairs, count, sizeof(FileImagePair), compareFileImagePairs);

    closedir(dir);

    //printf("HAHA\n");

        net = network_load("./neural_network", hexa);

    //printf("TEST : %d\n", count);

    for (int i = 0; i < count; i ++){
        Matrix* prediction = network_predict_img(net, fileImagePairs[i].img);
        int id = matrix_argmax(prediction);
        //printf("%s : %d, %.4lf\n", fileImagePairs[i].filename, id, matrix_score(prediction, id));
        int x = charToHex(fileImagePairs[i].filename[12]);
        int y = charToHex(fileImagePairs[i].filename[13]);
        res [x][y] = id;
        matrix_free(prediction);
    }

    for (int i = 0; i < count; i ++){
        img_free(fileImagePairs[i].img);
    }
    free(imgs);
    free(fileImagePairs);
    network_free(net);
}
```

3. **Comparaison avec l'étiquette réelle :**

- La fonction compare la classe prédite avec l'étiquette réelle de l'image en utilisant `matrix_argmax` pour obtenir l'indice de la classe avec la probabilité la plus élevée.

- Si l'indice prédit correspond à l'indice réel, le nombre de prédictions correctes (`n_correct`) est incrémenté.

4. **Affichage des résultats (débogage) :**

- La fonction affiche l'identifiant de la classe prédite (`id`) et la probabilité associée, utilisant `matrix_score`.

5. **Calcul du pourcentage de prédictions correctes :**

- La fonction retourne le pourcentage de prédictions correctes sur l'ensemble du lot, calculé en divisant le nombre de prédictions correctes par le nombre total d'images dans le lot.

En résumé, l'évaluation des performances du réseau de neurones se fait en mesurant la précision de ses prédictions sur un ensemble de données. Plus spécifiquement, la fonction `network_predict_imgs` effectue des prédictions sur un lot d'images, compare ces prédictions avec les étiquettes réelles, et retourne le pourcentage de prédictions correctes. Cette mesure est souvent utilisée comme indicateur de la qualité du modèle et de sa capacité à généraliser à de nouvelles données.

VII/ Interface Utilisateur(Thibaut et Théodore)

Conception avec Glade

La conception de l'interface graphique a été grandement simplifiée grâce à l'utilisation de Glade. Glade nous a permis de créer rapidement et efficacement des composants d'interface utilisateur en utilisant une approche de type glisser-déposer. Nous avons pu organiser les différents éléments de l'interface, tels que les boutons, les zones de dessin, et les indicateurs d'étapes, de manière intuitive.

Organisation de l'Interface

L'interface est structurée autour d'une fenêtre principale contenant plusieurs pages correspondant aux différentes étapes du processus de traitement d'image. Chaque page comporte des éléments spécifiques liés à l'étape en cours, offrant ainsi une expérience utilisateur cohérente et progressive.

Les boutons et les indicateurs sont connectés à des fonctions spécifiques dans le code, ce qui garantit une interactivité fluide et une réactivité à l'utilisateur. Par exemple, les boutons de rotation, de recadrage, et de résolution de sudoku sont tous liés à des fonctions distinctes qui sont exécutées en réponse aux actions de l'utilisateur.

Sélection de l’image à traiter

Sur la première page, nous trouvons un bouton qui permet de choisir avec quelle image l’utilisateur veut utiliser notre programme.

Rotation et Recadrage

L'interface propose des boutons de rotation à gauche et à droite, ainsi qu'une fonction de recadrage pour ajuster l'image. Ces fonctionnalités sont accompagnées d'une prévisualisation en temps réel, offrant à l'utilisateur un contrôle visuel sur les modifications apportées à l'image.

OCR et Résolution de Sudoku

La fonction OCR est déclenchée par un bouton spécifique, permettant à l'utilisateur de confirmer la reconnaissance optique des caractères. Nous avons également ajouté la possibilité de modifier les résultats trouvés par notre ocr en cas d'erreur. De même, la résolution de sudoku est initiée par un bouton distinct après le traitement de l'image, puis le résultat est affiché sur la dernière fenêtre, avec la possibilité d’enregistrer l’image finale ou de recommencer le processus.

Conclusion

L'utilisation de Glade pour la conception de l'interface graphique a significativement accéléré le processus de développement. L'interface résultante offre une expérience utilisateur ergonomique et intuitive, facilitant la navigation à travers les différentes étapes du traitement d'image.

Ce travail constitue une étape clé dans le développement de notre application, offrant aux utilisateurs une interface conviviale et efficace pour interagir avec les fonctionnalités de traitement d'image avancées.

Conclusion

Malgré les nombreuses difficultés rencontrées, nous avons réussi à accomplir proprement quasiment la totalité des étapes nécessaires à la réalisation d'un OCR Sudoku. Ce projet nous a permis d'acquérir de nombreuses compétences en ce qui concerne le réseau de neurones, le traitement d'image, la création d'une interface graphique,... Par conséquent, nous considérons ce projet comme une bonne expérience.